

Scenario: E-Commerce Company (ShopSmart)

Databases:

- `usersDB` → customer data
- `ordersDB` → order data
- `analyticsDB` → reporting data

Teams:

- App Developer
- Data Analyst
- DBA (Admin) ==

1. MongoDB Authentication — Scenario

Problem

Anyone can connect and delete data.

Solution: Enable authentication & create users

```
use admin
```

```
db.createUser({
  user: "adminUser",
  pwd: "Admin@123",
  roles: [ { role: "root", db: "admin" } ]
})
```

 Meaning:

This user can manage users in **all databases**.

atlas:

adminUser

Admin_001

2. Multiple Users & Role-Based Access (Scenario)

Requirement

User	Access
appUser	Read/Write only ordersDB
analystUser	Read-only analyticsDB
backupUser	Read all DBs

◆ Create App User

```
use ordersDB
db.createUser({
  user: "appUser",
  pwd: "App@123",
  roles: [ { role: "readWrite", db: "ordersDB" } ]
})
```

👉 Can insert/update orders

👉 Cannot access usersDB or analyticsDB

```
atlas
appUser
App_123
```

◆ Create Analyst User

```
use analyticsDB
db.createUser({
  user: "analystUser",
  pwd: "Analyst@123",
  roles: [ { role: "read", db: "analyticsDB" } ]
})
```

👉 Can only run find()

👉 Cannot insert/update

analyticalUser

◆ Create Backup User (Multiple DB access)

```
use admin
db.createUser({
  user: "backupUser",
  pwd: "Backup@123",
  roles: [
    { role: "read", db: "userDB1" },
    { role: "read", db: "ordersDB" },
  ]
})
```

```
    { role: "read", db: "analyticsDB" }  
  ]  
})
```

👉 Can read from 3 DBs

👉 Used for backup scripts

3. Roles Explained with Scenario

Role	Scenario Meaning
read	Reporting user
readWrite	Application user
dbAdmin	Index & schema management
userAdmin	Create users
root	Super admin

Example:

```
{ role: "dbAdmin", db: "ordersDB" }
```

👉 Can create indexes

👉 Cannot delete collections

4. IP Whitelisting — Scenario

❌ Problem

Hackers from random IP try to connect

✅ Solution: Allow only office IP

In Atlas → **Network Access**

Add:

203.0.113.10/32 (Office IP)

Meaning:

✓ Only this IP can connect

✓ All others blocked

5. Connection Strings — Scenario

App Developer needs connection string:

mongodb+srv://appUser:App@123@cluster0.mongodb.net/ordersDB

Analyst:

mongodb+srv://analystUser:Analyst@123@cluster0.mongodb.net/analyticsDB

Security benefit:

Different users → different permissions

6. Scaling, Resizing, Pause/Resume

Scenario 1: Festival Sale (High Traffic)

Orders increase 10x

Action in Atlas:

Cluster → Scale → Increase RAM/CPU

Result:

- ✓ Handles more users
- ✓ Faster queries

Scenario 2: Night Time (No Traffic)

Pause cluster

Action:

Pause Cluster

Result:

- ✓ Saves cost
- ✓ Resume when needed

7. Automated Backups & Restore — Scenario

Problem

Developer ran:

```
db.orders.drop()
```

 **Restore from backup**

Atlas → Backup → Restore → Select time

Prepare a document with restore and backup with one case study

Result:

- ✓ Data recovered
- ✓ Business continues

Backup user example:

```
db.createUser({
  user: "restoreUser",
  pwd: "Restore@123",
  roles: [ { role: "backup", db: "admin" } ]
})
```

8. Metrics (MongoDB Compass & Atlas)

Scenario: App slow today

In Atlas Metrics:

- CPU: 90%
- Connections: High
- Query Execution Time: High

Meaning:

👉 Need scaling or index

In MongoDB Compass

Open Performance tab

Observe:

- Slow queries
- Query shapes
- Index usage

Example:

```
db.orders.find({ userId: "U100" })
```

Compass shows:

✗ COLLSCAN

✓ After index → IXSCAN

```
db.orders.createIndex({ userId: 1 })
```

9. Privilege Granting on Multiple DBs

Scenario: Manager wants full control on reports

```
use admin
db.createUser({
  user: "reportManager",
  pwd: "Report@123",
  roles: [
    { role: "readWrite", db: "analyticsDB" },
    { role: "dbAdmin", db: "analyticsDB" }
  ]
})
```

Meaning:

- ✓ Insert reports
- ✓ Create indexes
- ✓ Manage collections
- ✗ Cannot touch ordersDB

Summary Mapping

Topic	Scenario
Authentication	Stop anonymous access
Users	Different staff → different powers
Roles	Fine-grained security
IP Whitelist	Block unknown networks
Connection Strings	Role-based DB access
Scaling	Handle load
Pause/Resume	Save money
Backups	Recover from mistakes
Metrics	Find performance issues

1. INSERT SCRIPTS (insertMany)

usersDB

```
use usersDB
```

```
db.customers.insertMany([
```

```

    { _id: 1, name: "Arun", email: "arun@gmail.com", city:
"Chennai" },
    { _id: 2, name: "Divya", email: "divya@gmail.com", city:
"Bangalore" },
    { _id: 3, name: "Rahul", email: "rahul@gmail.com", city:
"Hyderabad" }
  ])

db.admins.insertMany([
  { _id: 101, username: "admin1", role: "DBA" },
  { _id: 102, username: "admin2", role: "Support" }
])

```

ordersDB

```
use ordersDB
```

```

db.products.insertMany([
  { _id: "P100", name: "Laptop", price: 60000, stock: 10 },
  { _id: "P200", name: "Mobile", price: 30000, stock: 20 },
  { _id: "P300", name: "Headphones", price: 2000, stock: 50 }
])

db.orders.insertMany([
  {
    _id: "O1",
    customerId: 1,
    items: [
      { productId: "P100", qty: 1 },
      { productId: "P300", qty: 2 }
    ],
    totalAmount: 64000,
    orderDate: ISODate("2025-01-10")
  },
  {
    _id: "O2",
    customerId: 2,
    items: [
      { productId: "P200", qty: 1 }
    ],
    totalAmount: 30000,
    orderDate: ISODate("2025-01-12")
  },
  {
    _id: "O3",

```

```

    customerId: 1,
    items: [
      { productId: "P300", qty: 3 }
    ],
    totalAmount: 6000,
    orderDate: ISODate("2025-01-15")
  }
])

```

analyticsDB

```
use analyticsDB
```

```

db.salesReports.insertMany([
  { _id: 1, product: "Laptop", totalSold: 1, revenue:
60000 },
  { _id: 2, product: "Mobile", totalSold: 1, revenue:
30000 },
  { _id: 3, product: "Headphones", totalSold: 5, revenue:
10000 }
])

```

Logical Relationships

From	To	Relation
customers.customerId	orders.customerId	One-to-Many
orders.items.productId	products._id	Many-to-One
orders → salesReports	Aggregated result	
customers → userActivity	One-to-Many	

2. AGGREGATION PIPELINES

◆ A. Orders with Customer & Product Names

```
use ordersDB
```

```

db.orders.aggregate([
  {
    $lookup: {

```



```

        from: "usersDB.customers",
        localField: "customerId",
        foreignField: "_id",
        as: "customer"
    }
},
{ $unwind: "$items" },
{
    $lookup: {
        from: "products",
        localField: "items.productId",
        foreignField: "_id",
        as: "product"
    }
},
{
    $project: {
        orderId: "$_id",
        customer: { $arrayElemAt: ["$customer.name", 0] },
        product: { $arrayElemAt: ["$product.name", 0] },
        qty: "$items.qty",
        totalAmount: 1
    }
}
])

```

📌 Shows: who bought what and how much

◆ B. Total Quantity Sold Per Product

```

db.orders.aggregate([
    { $unwind: "$items" },
    {
        $group: {
            _id: "$items.productId",
            totalQty: { $sum: "$items.qty" }
        }
    }
])

```

◆ C. Revenue by Customer

```

db.orders.aggregate([
  {
    $group: {
      _id: "$customerId",
      totalSpent: { $sum: "$totalAmount" }
    }
  }
])

```

3. USER & ROLE CREATION (Mapped to DBs)

App User → ordersDB only

```

use ordersDB
db.createUser({
  user: "appUser",
  pwd: "App@123",
  roles: [ { role: "readWrite", db: "ordersDB" } ]
})

```

Analyst User → analyticsDB (read only)

```

use analyticsDB
db.createUser({
  user: "analystUser",
  pwd: "Analyst@123",
  roles: [ { role: "read", db: "analyticsDB" } ]
})

```

Backup User → all DBs (read)

```

use admin
db.createUser({
  user: "backupUser",
  pwd: "Backup@123",
  roles: [
    { role: "read", db: "usersDB" },
    { role: "read", db: "ordersDB" },
    { role: "read", db: "analyticsDB" }
  ]
})

```

Admin User → full control

```
use admin
db.createUser({
  user: "adminUser",
  pwd: "Admin@123",
  roles: [ { role: "root", db: "admin" } ]
})
```

4. BACKUP & RESTORE DEMO

Backup (Export)

```
mongoexport --db usersDB --collection customers --out
customers.json
mongoexport --db ordersDB --collection orders --out
orders.json
mongoexport --db analyticsDB --collection salesReports --out
salesReports.json
```

Simulate Data Loss

```
use ordersDB
db.orders.drop()
```

Restore (Import)

```
mongoimport --db ordersDB --collection orders --file
orders.json
```

5. PERFORMANCE TEST QUERIES

Without Index (Slow)

```
use ordersDB
db.orders.find({ customerId: 1 }).explain("executionStats")
Look for:
COLLSCAN
```

Create Index

```
db.orders.createIndex({ customerId: 1 })
```

After Index (Fast)

```
db.orders.find({ customerId: 1 }).explain("executionStats")
```

Look for:
IXSCAN
executionTimeMillis ↓

◆ Test Aggregation Performance

```
db.orders.aggregate([  
  { $match: { customerId: 1 } },  
  { $group: { _id: "$customerId", total: { $sum:  
"$totalAmount" } } }  
]).explain("executionStats")
```